

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет ИУ
Кафедра ИУ5

Курс «Парадигмы и конструкции языков программирования»

Отчет по лабораторной работе №6
«Разработка бота на основе конечного автомата для Telegram с использованием
языка Python»
Вариант №7

Выполнил:
студент группы ИУ5-32Б:
Гаранин Е. Д.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Гапанюк Ю. Е.
Подпись и дата:

Москва, 2025 г.

ОГЛАВЛЕНИЕ

1. Задание для лабораторной работы
2. Листинг кода
3. Пример выполнения

ЗАДАНИЕ ДЛЯ ЛАБОРАТОРНОЙ РАБОТЫ

Разработайте бота для Telegram. Бот должен реализовывать конечный автомат из трех состояний.

ЛИСТИНГ КОДА

fsm_bot.py:

```
import logging
from telegram import Update, ReplyKeyboardMarkup, ReplyKeyboardRemove
from telegram.ext import (
    Application,
    CommandHandler,
    MessageHandler,
    filters,
    ConversationHandler,
    ContextTypes,
)

# Включаем логирование для отладки
logging.basicConfig(
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    level=logging.INFO
)
logger = logging.getLogger(__name__)

# Определяем наши три состояния в виде констант
STATE_1, STATE_2, STATE_3 = range(3)

#
# ФУНКЦИИ-ОБРАБОТЧИКИ
#

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    """
    Точка входа в FSM.
    Активируется командой /start.
    """
    reply_keyboard = [['2', '3']]

    await update.message.reply_text(
        "Привет! Это бот-конечный автомат. 🤖\n"
        "Вы находитесь в Состоянии 1.\n\n"
        "Куда вы хотите перейти? (Отправьте '2' или '3').\n"
        "Или отправьте /cancel для выхода.",
        reply_markup=ReplyKeyboardMarkup(
            reply_keyboard, one_time_keyboard=True, resize_keyboard=True
        ),
        parse_mode='Markdown'
    )

    return STATE_1

async def handle_state_1(update: Update, context: ContextTypes.DEFAULT_TYPE)
-> int:
    """Обработчик для Состояния 1."""
    text = update.message.text

    if text == '2':
        reply_keyboard = [['3']]
        await update.message.reply_text(
            "☑ Переход 1 -> 2.\n"
            "Вы в Состоянии 2.\n"
            "Теперь вы можете перейти в Состояние 3 (отправьте '3').",
            reply_markup=ReplyKeyboardMarkup(
                reply_keyboard, one_time_keyboard=True, resize_keyboard=True
            ),
        ),
```

```

        parse_mode='Markdown'
    )
    return STATE_2

elif text == '3':
    reply_keyboard = [['1']]
    await update.message.reply_text(
        "☑ Переход 1 -> 3.\n"
        "Вы в Состоянии 3.\n"
        "Теперь вы можете перейти в Состояние 1 (отправьте '1').",
        reply_markup=ReplyKeyboardMarkup(
            reply_keyboard, one_time_keyboard=True, resize_keyboard=True
        ),
        parse_mode='Markdown'
    )
    return STATE_3

else:
    await update.message.reply_text(
        "✗ Неверный ввод.\n"
        "Вы все еще в Состоянии 1. Пожалуйста, введите '2' или '3'.",
        parse_mode='Markdown'
    )
    return STATE_1

async def handle_state_2(update: Update, context: ContextTypes.DEFAULT_TYPE)
-> int:
    """Обработчик для Состояния 2."""
    text = update.message.text

    if text == '3':
        reply_keyboard = [['1']]
        await update.message.reply_text(
            "☑ Переход 2 -> 3.\n"
            "Вы в Состоянии 3.\n"
            "Теперь вы можете перейти в Состояние 1 (отправьте '1').",
            reply_markup=ReplyKeyboardMarkup(
                reply_keyboard, one_time_keyboard=True, resize_keyboard=True
            ),
            parse_mode='Markdown'
        )
        return STATE_3
    else:
        await update.message.reply_text(
            "✗ Неверный ввод.\n"
            "Вы все еще в Состоянии 2. Пожалуйста, введите '3'.",
            parse_mode='Markdown'
        )
        return STATE_2

async def handle_state_3(update: Update, context: ContextTypes.DEFAULT_TYPE)
-> int:
    """Обработчик для Состояния 3."""
    text = update.message.text

    if text == '1':
        reply_keyboard = [['2', '3']]
        await update.message.reply_text(
            "☑ Переход 3 -> 1.\n"
            "Вы в Состоянии 1.\n"
            "Куда вы хотите перейти? (Отправьте '2' или '3').",
            reply_markup=ReplyKeyboardMarkup(

```

```

        reply_keyboard, one_time_keyboard=True, resize_keyboard=True
    ),
    parse_mode='Markdown'
)
return STATE_1
else:
    await update.message.reply_text(
        "❌ Неверный ввод.\n"
        "Вы все еще в Состоянии 3. Пожалуйста, введите '1'.",
        parse_mode='Markdown'
    )
    return STATE_3

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE) -> int:
    """
    Точка выхода из FSM.
    Активируется командой /cancel.
    """
    await update.message.reply_text(
        "Конечный автомат остановлен. 🟢\n"
        "Чтобы начать заново, введите /start.",
        reply_markup=ReplyKeyboardRemove()
    )

    return ConversationHandler.END

def main() -> None:
    """Основная функция запуска бота."""
    TOKEN = "YOUR_TOKEN"
    application = Application.builder().token(TOKEN).build()

    conv_handler = ConversationHandler(
        entry_points=[CommandHandler('start', start)],

        states={
            STATE_1: [MessageHandler(filters.TEXT & ~filters.COMMAND,
handle_state_1)],
            STATE_2: [MessageHandler(filters.TEXT & ~filters.COMMAND,
handle_state_2)],
            STATE_3: [MessageHandler(filters.TEXT & ~filters.COMMAND,
handle_state_3)],
        },

        fallbacks=[CommandHandler('cancel', cancel)],
    )

    application.add_handler(conv_handler)

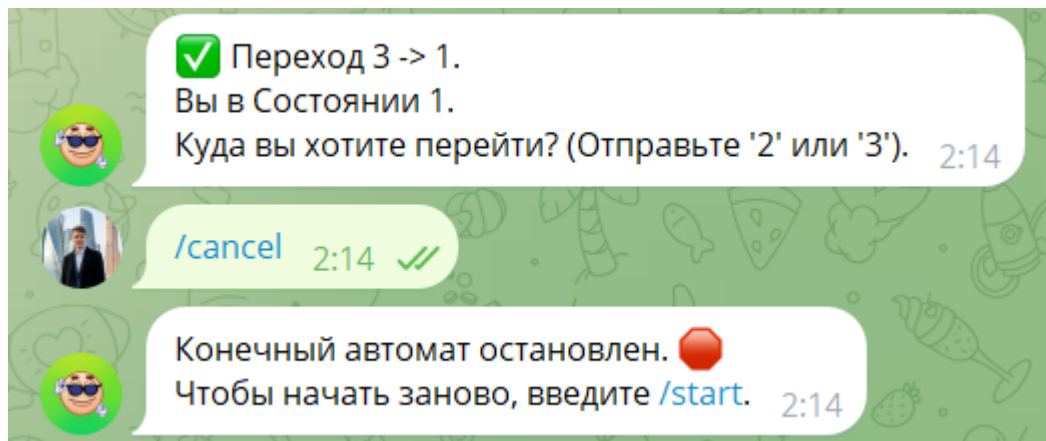
    logger.info("Бот запущен...")
    application.run_polling()

if __name__ == '__main__':
    main()

```

ПРИМЕР ВЫПОЛНЕНИЯ





Вывод: я изучил разработку ботов в Telegram.